

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents



01 Other important applications of Lists



02



03



04



05

List Comprehensions

- Python supports computed lists called “list comprehensions”

List1 = [*expression* for variable in sequence]

- Where, the expression is evaluated once, for every item in the sequence

List Comprehensions

- List1 = [*expression* for variable in sequence]
- List comprehensions help programmers to create lists in a concise way

List Comprehensions

Example Program to make a list of cubes of numbers

```
cubes = [] # an empty list
for i in range(11):
    cubes.append(i**3)
print("Cubes of numbers from 1-10 : ", cubes)
```

OUTPUT

```
Cubes of numbers from 1-10 :  [0, 1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

List Comprehensions

Example Program to make a list of cubes of numbers using List Comprehension.

```
L = [x**3 for x in range (11)]
```

Looping in Lists

- Python's "for" and "in" constructs are extremely useful especially when working with lists
- The "**for item in list**" statement is an easy way to access each element in a list (or any other sequence)

Looping in Lists

- Example,

```
L = [1, 2, 3, 4, 5]
```

```
for item in L:
```

```
    print(item)
```


Looping in Lists

```
num_list = [1,2,3,4,5,6,7,8,9,10]
sum = 0
for i in num_list:
    sum += i
print("Sum of elements in the list = ", sum)
print("Average of elements in the list = ",
      float(sum/float(len(num_list))))
```

OUTPUT

```
Sum of elements in the list =  55
Average of elements in the list =  5.5
```

Looping in Lists enumerate()

- **enumerate()** function is used when you want to print both index as well as an item in the list.
- The function returns an enumerate object which contains the index and value of all the items of the list as a tuple.

Looping in Lists enumerate()

```
num_list = [1,2,3,4,5]
for index, i in enumerate(num_list):
    print(i, " is at index : ", index)
```

OUTPUT

```
1  is a t index :  0
2  is a t index :  1
3  is a t index :  2
4  is a t index :  3
5  is a t index :  4
```

Looping in Lists enumerate()

- The **range()** function is used when you need to print index.

Looping in Lists range()

#To print index using range() function

```
num_list = [1,2,3,4,5]
for i in range(len(num_list)):
    print("index : ", i)
```

OUTPUT

```
index : 0
index : 1
index : 2
index : 3
index : 4
```

Using an Iterator

- You can create an iterator using the built-in **iter()** function.
- The iterator is used to loop over the elements of the list.

Using an Iterator

- For this, the iterator fetches the value and then automatically points to the next element in the list when it is used with the **next()** method.

Using an Iterator

```
num_list = [1,2,3,4,5]
it = iter(num_list)
for i in range(len(num_list)):
    print("Element at index ", i, " is : ", next(it))
```

OUTPUT

```
Element at index 0 is : 1
Element at index 1 is : 2
Element at index 2 is : 3
Element at index 3 is : 4
Element at index 4 is : 5
```


filter() Function

- The **filter() function** constructs a list from those elements of the list for which a function returns True.
- The syntax of the filter() function is given as, **filter(function, sequence)**

filter() Function

- As per the syntax, the **filter()** function returns a sequence that contains items from the sequence for which the function is True.

filter() Function

- If sequence is a string, Unicode, or a tuple, then the result will be of the same type;
- otherwise, it is always a list.

filter() Function

Make a list of Even Numbers from a Sequence of Numbers

Example:

```
evens = list(filter(even, range(1, 11)))
```

filter(function, sequence)

Make a list of Even Numbers from a Sequence of Numbers

```
def even(n):
```

```
    if n%2==0:
```

```
        return(1)
```

```
evens = list(filter(even, range(1, 11)))
```

```
print(evens)
```

map() Function

- The `map()` function applies a particular function to every element of a list. Its syntax is same as the filter function

```
map(function, sequence)
```

map() Function

- After applying the specified function on the sequence, the `map()` function returns the modified list.
- The `map()` function calls `function(item)` for each item in the sequence and returns a list of the return values.

map() Function

- Example: Program that adds 2 to every value in the list

```
def add_2(x):  
    x += 2  
    return x  
num_list = [1,2,3,4,5,6,7]  
print("Original List is : ", num_list)  
new_list = list(map(add_2, num_list))  
print("Modified List is : ", new_list)
```

OUTPUT

```
Original List is :  [1, 2, 3, 4, 5, 6, 7]  
Modified List is :  [3, 4, 5, 6, 7, 8, 9]
```


reduce() Function

- The reduce() function with syntax as given below returns a single value generated by calling the function
- on the first two items of the sequence, then on the result and the next item, and so on.

```
reduce(function, sequence)
```

reduce() Function

- Example: Program to calculate the sum of values in a list using the reduce() function

```
import functools #functools is a module that contains the function reduce()
def add(x,y):
    return x+y
num_list = [1,2,3,4,5]
print("Sum of values in list = ")
print(functools.reduce(add, num_list))
```

OUTPUT

Sum of values in list = 15

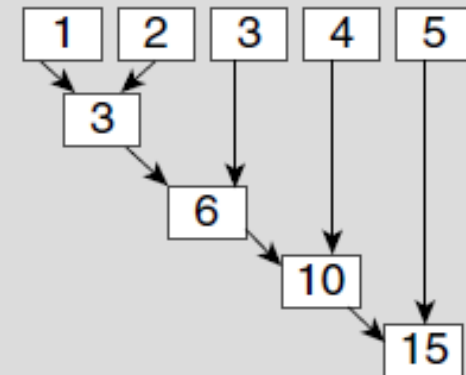


Figure 8.4 reduce() function

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025